

Back To Practice

Save

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

Select Language : C (GCC 9.2.0)

```

1 #include <stdio.h>
2 int main() {
3     int m, n;
4     printf("Enter number of memory blocks:\n");
5     scanf("%d", &m);
6     int blockSize[m];
7     int original[m];
8     printf("Enter size of each block:\n");
9     for(int i = 0; i < m; i++) {
10         scanf("%d", &blockSize[i]);
11         original[i] = blockSize[i];
12     }
13     printf("Enter number of processes:\n");
14     scanf("%d", &n);
15     int processSize[n];
16     printf("Enter size of each process:\n");
17     for(int i = 0; i < n; i++) {
18         scanf("%d", &processSize[i]);
19     }
20 }

```

Test Cases 2

Run Sample Cases Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Back To Practice

Q1

Save

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

Select Language : C (GCC 9.2.0)

```

19 }
20 for(int i = 0; i < n; i++) {
21     int bestIndex = -1;
22     for(int j = 0; j < m; j++) {
23         if(blockSize[j] >= processSize[i]) {
24             if(bestIndex == -1 || blockSize[j] < blockSize[bestIndex]) {
25                 bestIndex = j;
26             }
27         }
28     }
29     if(bestIndex != -1) {
30         int fragment = blockSize[bestIndex] - processSize[i];
31         printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
32             i + 1, processSize[i], bestIndex + 1,
33             original[bestIndex], fragment);
34         blockSize[bestIndex] -= processSize[i];
35     }
36     else {

```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Back To Practice

Save

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

Memory block allocation: Memory block number, block size, and process

Select Language : C (GCC 9.2.0)

```
24 if (bestIndex == -1 || blockSize[j] < blockSize[bestIndex]) {
25     bestIndex = j;
26 }
27 }
28 }
29 if (bestIndex != -1) {
30     int fragment = blockSize[bestIndex] - processSize[i];
31     printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
32         i + 1, processSize[i], bestIndex + 1,
33         original[bestIndex], fragment);
34     blockSize[bestIndex] -= processSize[i];
35 }
36 else {
37     printf("Process %d of size %d is not allocated\n",
38         i + 1, processSize[i]);
39 }
40 }
41 return 0;
42 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1